

The Unity Editor Enhanced: A Comprehensive Guide to the KUBACHO LAB 'v-Series' and the Art of Custom Tool Development

Introduction: Beyond the Default - The Philosophy of a Streamlined Workflow

The Problem of Friction in Unity Development

The default Unity Editor is a remarkably powerful and flexible environment for game development. However, as projects grow in scale and complexity, developers often encounter subtle but persistent points of friction. This friction manifests as cognitive load—the mental effort required to parse a sprawling hierarchy, the repetitive clicks needed to navigate the Project window, or the time lost to minor but frequent interruptions while interacting with the Inspector. While no single action is debilitating, their cumulative effect can significantly impede productivity, creativity, and overall development velocity. This is the challenge that a class of "quality of life" editor tools aims to solve: not by adding revolutionary new engine features, but by sanding down the rough edges of the daily workflow.

Introducing KUBACHO LAB

Emerging from this need for a smoother development experience is KUBACHO LAB, a publisher on the Unity Asset Store founded by developer Maxim Petrochenko.¹ The studio's

guiding principle is simple yet profound: "Make Unity feel like home".² This ethos is embodied in their flagship suite of editor enhancement tools, collectively known as the "v-series." These packages are not disparate utilities but a cohesive ecosystem designed to address specific pain points within Unity's core windows: the Hierarchy, the Inspector, the Project window, and the Scene view.

The 'Pragmatic Enhancement' Philosophy

To fully appreciate the v-series, it is essential to understand the philosophy that underpins its design. These tools were not conceived in a vacuum; they are the product of years of direct experience with the Unity Editor's limitations. The developer, "kubacho-lab," has noted spending seven years working in Unity without such tools before identifying the specific workflow bottlenecks that needed fixing.⁴ This history informs a design approach that can be described as "Pragmatic Enhancement."

The goal is not to create the most feature-dense or powerful tools on the market, a niche occupied by comprehensive systems like Odin Inspector. Instead, the v-series focuses on providing intuitive, lightweight, and seamlessly integrated improvements. Across user reviews and official descriptions, the assets are consistently praised for being "minimal," "intuitive," "lightweight," and free of "clutter" or "intrusive features".⁵ The developer explicitly positions vInspector 2 as a tool for user experience (UX) improvements that Odin lacks, rather than a tool for building highly complex custom editors.⁸ This deliberate focus results in a suite that solves the most common 80% of daily workflow issues with a 20% solution that is performant, easy to adopt, and works cohesively across the entire editor.

Overview of the Suite and Versioning

The v-series consists of six primary tools, each targeting a different aspect of the editor. Many of these tools exist in both a legacy version 1 and a significantly upgraded version 2. This report will focus on the modern version 2 releases, which introduce critical features and refinements.⁶ The developer's continued support for version 1 users demonstrates a commitment to their customer base and a stable development practice.⁸ The following table provides a high-level overview of the suite's components and their interactions.

Tool Name	Target Editor	Core Purpose	Key Features	Key Integration
-----------	---------------	--------------	--------------	-----------------

	Window(s)			Points
vHierarchy 2	Hierarchy	Enhances visual organization and interaction with scene objects.	Custom icons & colors, component minimap, hierarchy lines, shortcuts.	Icons appear in vFavorites 2, vTabs 2, and vInspector 2.
vInspector 2	Inspector	Streamlines component management and inspector layout.	Component pinning, enhanced copy/paste, save in play mode, attributes.	Can display icons set by vHierarchy 2.
vFolders 2	Project	Adds visual identifiers to project folders.	Custom icons and colors for folders.	Icons appear in vFavorites 2 and vTabs 2.
vFavorites 2	Project, Hierarchy	Provides a quick-access bookmarking system for any asset or object.	On-demand popup window, paged organization, shortcuts.	Displays icons set by vHierarchy 2 and vFolders 2.
vTabs 2	Inspector, Project	Implements a browser-like tabbing system for editor windows.	Drag-and-drop tab creation, browser shortcuts, favorite layouts.	Displays icons set by vHierarchy 2 and vFolders 2.
vRuler	Scene	Offers precision measurement and alignment tools.	Distance, dimension, and angle measurement in the scene	Standalone; primarily a scene view utility.

			view.	
--	--	--	-------	--

Part I: The 'v-Series' In-Depth Guide

Chapter 1: vHierarchy 2 - Mastering the Scene Outline

Purpose and Value Proposition

In any non-trivial Unity scene, the Hierarchy window can quickly become a monotonous, hard-to-parse list of GameObjects. Identifying key systems like managers, UI canvases, or player prefabs requires careful reading and mental mapping. vHierarchy 2 directly addresses this by transforming the Hierarchy from a simple list into a rich, color-coded, and interactive scene outline, dramatically reducing the time spent searching for objects.

Core Functionality Deep Dive

- **Visual Organization:** The cornerstone of vHierarchy 2 is its powerful visual customization. By holding the Alt key and clicking on a GameObject, a customization pop-up appears.¹² From here, users can assign one of many built-in icons and apply a background color to the entry. This allows for the creation of a consistent visual language across projects—for example, always coloring manager objects red and UI elements blue.¹² The asset also adds subtle but effective readability aids like zebra striping (alternating row colors) and hierarchy lines that visually connect parent and child objects.⁶ For more complex visual organization, empty GameObjects can be styled as dividers to break the hierarchy into logical sections.¹²
- **Component Minimap:** To the right of each GameObject, vHierarchy 2 displays a "minimap" of its attached components as small icons. This provides an immediate, at-a-glance understanding of what an object does without needing to select it and inspect its components. For quick edits, Alt-clicking a component icon opens a "mini

editor" in a pop-up window, allowing for adjustments without losing the current Inspector selection.⁶

- **Enhanced Interaction:** A significant workflow enhancement is the ability to interact with objects directly in the hierarchy without selecting them first. By simply hovering the mouse over an entry, a suite of shortcuts becomes available: press A to toggle the object's active state, F to focus the Scene view camera on it, X to delete it, and E to expand or collapse its children. Shift-E provides a "solo" expand, collapsing all other sibling objects.⁶ These shortcuts eliminate thousands of clicks over the course of a project.
- **Customization and Palettes:** The palette of available icons and colors is not fixed. Users can add their own custom icons to the selection list, allowing for project-specific visual identifiers.⁶ Furthermore, these customized palettes can be exported and shared among team members or used in other projects, ensuring a consistent organizational scheme.⁶ For fine-grained control, the developer has noted that additional colors can be added to the list by selecting the palette's data object and using the Inspector's debug mode.⁹

Advanced Techniques and Synergies

- **Prefab Workflow:** A key improvement in version 2 is the ability to set icons and colors directly within Prefab Mode. These visual settings are then automatically applied to all instances of that prefab placed in a scene, ensuring that, for example, all "Enemy" prefabs are instantly recognizable.⁶
- **Team Collaboration:** By default, vHierarchy stores its data in a central ScriptableObject. However, for teams using version control, this can lead to merge conflicts. The asset provides an option to store customization data directly within the scene files themselves, which is often easier for teams to manage.⁶
- **Integration:** vHierarchy 2 serves as the visual foundation for the rest of the suite. The icons and colors assigned here are automatically recognized and displayed by vFavorites 2, vTabs 2, and vInspector 2, creating a unified and consistent visual language that permeates the entire editor.⁵

Under the Hood: Technical Insights

The functionality of vHierarchy 2 is achieved through clever use of specific Unity Editor APIs designed for extension. The core of the tool is almost certainly built upon the `EditorApplication.hierarchyWindowItemOnGUI` delegate. This is a special event that Unity

invokes for every visible item in the Hierarchy window, providing two key arguments: the instance ID of the GameObject being drawn and a Rect defining the screen space it occupies.

Within a method subscribed to this delegate, vHierarchy 2 can perform all of its drawing operations. It can draw a colored rectangle as a background for zebra striping or custom colors. It can then draw the custom icons, the component minimap, and the hierarchy lines within the provided Rect. This delegate also allows the script to process mouse events, such as detecting an Alt-Click to open the customization menu or a hover event to listen for keyboard shortcuts. This approach is highly performant because the code only executes for items that are currently visible on screen, which explains the asset's claim of having "no lags or freezes".⁶ The settings, colors, and icon palettes are stored persistently in a

ScriptableObject, which is loaded into memory when the editor starts, a process likely initiated by a script with the [InitializeOnLoad] attribute.

Chapter 2: vInspector 2 - Reinventing Component Interaction

Purpose and Value Proposition

The default Unity Inspector, while functional, can be verbose and static. Long lists of public variables become difficult to manage, and performing common actions requires navigating menus or writing boilerplate code. vInspector 2 addresses this by overhauling the Inspector's UI and providing a powerful, attribute-based system for creating clean, interactive, and highly functional custom inspectors without writing a single line of custom editor code.

Core Functionality Deep Dive

- **UI/UX Enhancements:** vInspector 2 introduces several immediate improvements. A navigation bar at the top of the Inspector allows for bookmarking objects and navigating back and forth through selection history.⁵ Components can be Alt-Dragged out of the Inspector to "pin" them in their own floating window, invaluable for comparing components or keeping a reference visible.⁵ The UI is further polished with smooth animations for collapsing and deleting components, and a "Minimal Mode" can be enabled to hide unnecessary buttons and reduce visual clutter.⁵

- **Component Management:** The asset significantly enhances component workflows. It allows for copying multiple components at once and provides distinct options to "Paste as new" or "Paste values" onto existing components.⁵ One of its most celebrated features is the ability to save component changes made during Play Mode. Small "save" buttons appear on components while the game is running, allowing developers to persist tweaks and new values that would normally be lost when exiting Play Mode.⁵
- **Shortcuts:** A range of keyboard shortcuts speeds up interaction. Ctrl-Shift-E collapses or expands all components on a GameObject, while Shift-E solos a single component. The A key toggles the selected component's enabled state, and X deletes it.⁵
- **The Attribute System:** The true power of vInspector 2 lies in its attribute system. By adding simple tags above variables or methods in a C# script, developers can radically alter how they are displayed in the Inspector.⁵ Key attributes include:
 - ````: Turns a method into a clickable button in the Inspector. Version 2 enhances this with support for arguments and custom button colors.⁵
 - `[Foldout]` and ````: Organizes groups of variables into collapsible sections or tabs, cleaning up long component scripts.
 - ````: Exposes private, protected, or static variables to the Inspector without making them public, adhering to good encapsulation practices.
 - `[OnValueChanged]`: Specifies a method to be called automatically whenever the decorated variable's value is changed in the Inspector.
 - `, `[HideIf]`, :` Provide conditional logic for displaying and editing variables based on the state of other fields.
 - `[Variants]`: Creates a simple dropdown or button group for a variable, allowing developers to choose from a predefined set of values.
- **Dictionary Serialization:** A notorious limitation of Unity's default serializer is its inability to handle dictionaries. vInspector 2 provides out-of-the-box support for serializing and editing dictionaries directly in the Inspector, a massive quality-of-life improvement for data-driven design.⁵

Under the Hood and The Odin Comparison

A common point of comparison for vInspector 2 is the popular Odin Inspector asset. While they appear to solve similar problems, their underlying technical approaches are fundamentally different, leading to distinct trade-offs in performance, complexity, and stability.

Odin Inspector's core feature is its powerful, proprietary "Odin Serializer." It completely replaces Unity's serialization system for objects that inherit from its `SerializedMonoBehaviour`. This allows it to serialize virtually anything, including complex polymorphic types and

generics.¹⁶ This is a deep, foundational change that grants immense power at the cost of higher performance overhead and a greater risk of compatibility issues between Unity versions.⁴

vInspector 2, in contrast, is a highly advanced custom *renderer* that works *with* Unity's native serialization system. Its attribute system is implemented via a single, global CustomEditor script that targets UnityEngine.Object. When drawing the Inspector for any component, this script uses C# reflection to scan the object for vInspector's attributes. It then dynamically builds the Inspector UI using standard EditorGUILayout calls, drawing buttons for attributes, creating tabs for attributes, and so on, instead of just calling the default DrawDefaultInspector() method.

Its dictionary support is likely achieved by providing a custom SerializableDictionary class that implements Unity's ISerializationCallbackReceiver interface. This interface allows the class to convert its dictionary into a Unity-friendly format (like two lists for keys and values) before serialization and convert it back after deserialization. The custom Inspector then knows how to interpret these two lists and draw them as a user-friendly dictionary. This approach extends Unity's serializer rather than replacing it, which is the key to its lightweight nature and stability.⁵

This philosophical and technical divide makes neither tool strictly "better" than the other; they are suited for different purposes. The following table clarifies this distinction.

Comparison Axis	vInspector 2	Odin Inspector
Core Philosophy	Pragmatic Enhancement: Lightweight, intuitive UX improvements.	Ultimate Power: Full serialization replacement for maximum flexibility.
Primary Use Case	Quickly cleaning up inspectors and adding simple interactivity without custom code.	Building complex, data-heavy custom editors and serializing complex data structures.
Performance Overhead	Low. Uses reflection at draw time but leverages the native serializer.	Higher. Involves a custom serialization pass on top of Unity's. ¹⁷
Serialization	Extends Unity's serializer via	Replaces Unity's serializer for derived types.

	ISerializationCallbackReceiver.	
Attribute Complexity	Smaller, more focused set of intuitive attributes. ⁸	Vast library of over 100 attributes for fine-grained control. ¹⁶
Ease of Use	Very high. Attributes are simple and require no special setup.	Moderate. More powerful but has a steeper learning curve.
Learning Curve	Low. Can be adopted in minutes.	Moderate to High. Requires understanding its serialization model.
Licensing Model	Standard one-time purchase per seat on the Asset Store.	Complex, with different tiers and potential subscription components. ⁴

Chapter 3: vFolders 2 & vFavorites 2 - Taming the Project Window

Purpose and Value Proposition

As a project accumulates thousands of assets, the Project window can become as difficult to navigate as a complex scene hierarchy. Finding specific folders or frequently accessed prefabs can devolve into a tedious process of scrolling and searching. vFolders 2 and vFavorites 2 work in tandem to solve this problem by introducing visual organization and a rapid-access bookmarking system.

vFolders 2 Functionality

The premise of vFolders 2 is simple but effective: it allows developers to assign custom icons

and background colors to folders in the Project window.¹⁹ This enables the creation of a visual hierarchy that makes it easy to identify critical folders—such as "Prefabs," "Scripts," or "Art"—at a glance. This visual information is not confined to the Project window; the custom folder icons are also displayed in other tools like vFavorites 2 and vTabs 2, reinforcing the organizational scheme across the editor.¹¹ All customization data is stored in a

ScriptableObject located in a dedicated Assets/vFolders/ directory. For teams using version control, this folder can be added to the .gitignore file to allow each developer to maintain their own local folder appearance without creating conflicts.⁷

vFavorites 2 Functionality

vFavorites 2 provides a powerful bookmarking system that goes beyond just folders. Users can drag and drop any asset, folder, or even a GameObject from the scene into the vFavorites window to create a persistent shortcut.¹³ The tool is designed with a minimal footprint; the favorites window is not always visible. Instead, it appears as an overlay only when the

Alt key is held down while the mouse is over the Project window, ensuring it never permanently occupies valuable screen real estate.¹³ For better organization, bookmarks can be grouped into multiple pages. Navigation is streamlined with shortcuts: the mouse scroll wheel can be used to cycle through pages, and the

1-9 number keys provide direct access to the first nine pages.¹³

Under the Hood: Technical Insights

The technical implementation of these two tools mirrors the approach used by vHierarchy. vFolders 2 almost certainly uses the `EditorApplication.projectWindowItemOnGUI` delegate. This hook functions identically to its hierarchy counterpart but operates on items in the Project window. Within this delegate, the script can draw color overlays and custom icons on top of the default folder GUI.

The vFavorites 2 window is a custom `EditorWindow`. Its clever, on-demand activation is likely managed by a script with the `[InitializeOnLoad]` attribute. This script would subscribe to the `EditorApplication.update` event, which is called many times per second. Inside the update loop, the script would check the current event for the state of the Alt key and the position of the mouse. If the Alt key is pressed and the mouse cursor is within the bounds of the Project

window, it would call the method to show the EditorWindow. When the key is released or the mouse moves away, the window would be closed. This event-driven polling is an efficient way to create context-sensitive, non-intrusive UI.

Chapter 4: vTabs 2 - A Browser-Like Experience in Unity

Purpose and Value Proposition

Managing multiple Inspector or Project windows in Unity often leads to a cluttered workspace of floating or docked panels. Switching between a character prefab, an environment asset, and a manager object can require multiple clicks and selections. vTabs 2 streamlines this entire process by introducing a familiar, web browser-style tabbing system to Unity's core windows.

Core Functionality Deep Dive

vTabs 2 allows users to create tabs for nearly anything. Folders, assets, GameObjects, and even individual components can be dragged and dropped onto a window to create a new tab.¹¹ This allows a developer to, for example, have one Inspector window with tabs for the Player, the GameManager, and the main UI Canvas, and switch between them with a single click.

The interaction model is intentionally designed to mimic modern web browsers, making it instantly intuitive. Shift-Scroll cycles through open tabs, while Ctrl-Shift-Scroll reorders them. Standard shortcuts like Ctrl-T to add a new tab, Ctrl-W to close the current tab, and Ctrl-Shift-T to reopen the last closed tab are all supported.¹¹ For frequently used workflows, a set of tabs can be saved as a favorite layout and recalled later, instantly setting up a workspace for a specific task.¹¹

Under the Hood: Technical Insights

The implementation of vTabs 2 is a particularly insightful example of advanced editor scripting. A revealing comment from the developer provides the key to its architecture: "There's no other way to reliably implement folder tabs than by using locked project tabs, as that's the only way to mimic project window functionality and receive necessary callbacks".⁸

This statement implies that vTabs 2 does not create a new, custom type of window from scratch. Instead, it manipulates Unity's existing native windows. When a user creates a "folder tab," the asset likely performs the following steps programmatically:

1. It creates a new, standard Project window instance.
2. It uses editor scripting to "lock" this new window, which restricts its view to a specific folder.
3. It then hides the native window frame and overlays its own custom GUI for the tab bar at the top.

The asset maintains a collection of these hidden, locked windows, and when a user clicks on a tab, it simply brings the corresponding pre-configured native window to the front. This is a complex but brilliant solution. It ensures that all native Project window functionality—drag-and-drop, asset import callbacks, right-click context menus—works perfectly, because the user is, in fact, interacting with a real Project window. A similar technique would be used for Inspector tabs, where each tab corresponds to a locked Inspector window. This approach guarantees maximum compatibility and robustness by leveraging the native editor components instead of attempting to replicate them.

Chapter 5: vRuler - Precision Tools for Scene Layout

Purpose and Value Proposition

While Unity's grid and snapping tools are useful, they often fall short for tasks requiring high-precision measurements and alignments, such as architectural visualization, detailed level design, or complex UI placement in world space. vRuler fills this gap by providing a suite of measurement tools that operate directly within the Scene View.

Core Functionality Deep Dive

As a dedicated measurement utility, vRuler's primary function is to quantify spatial relationships within a scene.¹⁹ While detailed feature lists are less available compared to other v-series assets, its functionality can be reliably extrapolated from its purpose and the common requirements of such tools. Expected features include:

- **Point-to-Point Measurement:** Clicking two points in the scene to display the precise distance between them.
- **Object Dimensions:** Selecting an object to view the dimensions of its bounding box along the X, Y, and Z axes.
- **Alignment Guides:** Creating temporary lines or planes to help align multiple objects.
- **Angle Measurement:** Selecting three points to measure the angle between the resulting vectors.

These tools are indispensable for ensuring that game worlds adhere to design specifications and maintain a consistent scale.

Under the Hood: Technical Insights

The technical foundation for any tool that draws custom interactive elements in the Scene View is the combination of the `SceneView.duringSceneGui` delegate and the `UnityEditor.Handles` class. The `duringSceneGui` event is called during the Scene View's drawing loop, allowing scripts to inject their own rendering and input handling.

The `Handles` class provides a rich API for drawing 3D elements and controls within this context. vRuler would use this API extensively. For example, to implement point-to-point measurement, the script would:

1. Subscribe a method to `SceneView.duringSceneGui`.
2. Inside this method, listen for mouse clicks to capture 3D world-space coordinates for the ruler's start and end points.
3. Use `Handles.DrawLine()` to draw a visual line between these two points.
4. Calculate the distance using `Vector3.Distance()`.
5. Use `Handles.Label()` to display the calculated distance as text next to the line in the Scene View.

Interactive elements, like draggable ruler endpoints, would be created using controls like `Handles.PositionHandle()`. This combination of a drawing delegate and the `Handles` API is the standard, performant, and correct way to build custom in-scene editing tools in Unity.

Part II: Architecting Your Own Editor Enhancements

Chapter 6: Foundations of Unity Editor Scripting

The 'Editor' Folder

The gateway to extending the Unity Editor is a special folder named "Editor". Any C# script placed inside a folder with this name (at any level in the Assets directory) is treated as an Editor Script. This has two critical implications: the scripts are not included in the final game build, and they gain the ability to reference the UnityEditor namespace, which contains all the APIs for customizing the editor itself.²³

The UnityEditor Namespace

This namespace is the toolbox for building custom tools. A comprehensive understanding of its key classes is essential²⁶:

- Editor: The base class for creating custom inspectors for a specific component type.
- EditorWindow: The base class for creating new, dockable windows like the Hierarchy or Inspector.
- PropertyDrawer: Used to customize the GUI for a specific serializable class or a field with a custom attribute.
- MenuItem: An attribute that adds a function to the main Unity menu bar, creating an entry point for a tool.
- AssetDatabase: A static class for interacting with the project's assets—finding, loading, creating, and modifying them via code.
- EditorGUI & EditorGUILayout: These classes contain the functions for drawing all the GUI controls (labels, buttons, sliders, etc.) in a custom inspector or window. EditorGUILayout handles automatic layout, while EditorGUI requires manual Rect positioning.
- Handles: The class for drawing custom interactive GUI elements within the 3D Scene View.

Essential Concepts

- **MenuItem:** The `MenuItem` attribute is placed above a static method. When the corresponding menu item is clicked, Unity executes that method. This is the primary way to launch custom editor windows or run standalone editor functions.²⁵
- **SerializedObject & SerializedProperty:** Directly modifying a component's public variables from an editor script is a common mistake. This approach bypasses crucial Unity systems like Undo/Redo, multi-object editing, and prefab value handling. The correct method is to work with a `SerializedObject` representation of the component. A `SerializedObject` wraps the target object(s) and allows modification through `SerializedProperty` objects. When changes are applied via `serializedObject.ApplyModifiedProperties()`, Unity automatically handles undo, prefab overrides, and applying the change to all selected objects.

Best Practices for Editor Code

- **Performance:** Editor scripts, especially those hooking into OnGUI or update delegates, run frequently. Avoid allocating memory (new) or performing heavy computations inside these loops to keep the editor responsive.
- **Robustness:** Always use the `SerializedObject` and `SerializedProperty` system for modifying component data to ensure proper integration with the editor's features.
- **User Experience:** Strive for a native look and feel by using `EditorGUILayout` and Unity's built-in GUI styles. Provide clear labels, tooltips, and feedback to make your tools intuitive for other team members.

Chapter 7: Building a Custom Inspector with Attributes

Goal

This chapter aims to demystify the attribute-based approach of `Inspector 2` by guiding the

user through the creation of their own simplified versions of the and attributes.

PropertyAttribute and PropertyDrawer

These two components are the foundation for custom field drawing. A PropertyAttribute is a simple class that acts as a "tag" you can place on a serialized field in your script. A PropertyDrawer is a class that tells Unity how to draw the GUI for any field that has been tagged with a specific PropertyAttribute. Unity automatically finds the correct drawer for an attribute and uses it instead of the default field drawer.²⁸

Project: Recreating and

1. `` Attribute

First, create the attribute itself. This is just a marker and contains no logic.

C#

```
// Place in a regular script file (not in an Editor folder)
```

```
using UnityEngine;
```

```
public class ReadOnlyAttribute : PropertyAttribute { }
```

Next, create the PropertyDrawer that defines the behavior for this attribute. This script *must* be in an "Editor" folder.

C#


```
// Place in an "Editor" folder
using UnityEngine;
using UnityEditor;

public class ReadOnlyDrawer : PropertyDrawer
{
    public override void OnGUI(Rect position, SerializedProperty property, GUIContent label)
    {
        // Disable the GUI so it's not interactable
        GUI.enabled = false;

        // Draw the default property field, which will now be greyed out
        EditorGUI.PropertyField(position, property, label, true);

        // Re-enable the GUI for subsequent fields
        GUI.enabled = true;
    }
}
```

Now, in any MonoBehaviour, you can use `` above a serialized field to make it non-editable in the Inspector. This project demonstrates how a PropertyDrawer can modify the GUI state before drawing a control.

2. `` Attribute

The `` attribute is more complex as it doesn't modify a field but rather targets a method. This requires a custom Editor script rather than a PropertyDrawer.

First, define the attribute.

C#

```
// Place in a regular script file
using System;
```

```
public class ButtonAttribute : Attribute { }
```

Next, create a generic custom Editor script that will apply to all MonoBehaviour scripts. This script uses reflection to find and draw the buttons.

C#

```
// Place in an "Editor" folder
```

```
using UnityEngine;
```

```
using UnityEditor;
```

```
using System.Reflection;
```

```
public class ButtonEditor : Editor
```

```
{
```

```
    public override void OnInspectorGUI()
```

```
    {
```

```
        // Draw the default inspector first
```

```
        base.OnInspectorGUI();
```

```
        // Get the target script instance
```

```
        var monoBehaviour = (MonoBehaviour)target;
```

```
        // Find all methods in the script
```

```
        var methods = monoBehaviour.GetType()
```

```
            .GetMethod(BindingFlags.Instance | BindingFlags.Static | BindingFlags.Public |  
BindingFlags.NonPublic);
```

```
        foreach (var method in methods)
```

```
        {
```

```
            // Check if the method has our ButtonAttribute
```

```
            var buttonAttribute = method.GetCustomAttribute<ButtonAttribute>();
```

```
            if (buttonAttribute != null)
```

```
            {
```

```
                // Draw a button with the method's name
```

```
                if (GUILayout.Button(method.Name))
```

```
                {
```

```
                    // If the button is clicked, invoke the method
```

```
                    method.Invoke(monoBehaviour, null);
```

```
                }
```

```
}  
}  
}  
}
```

This project perfectly illustrates the core technique behind attribute-based inspectors like `vInspector`. It uses reflection to discover metadata (the attributes) and dynamically alters the Inspector GUI based on that metadata.

Chapter 8: Creating a Custom Editor Window

Goal

This chapter provides the skills to create custom, dockable tool windows, taking inspiration from the functionality of `vFavorites 2`.

The EditorWindow Class

An `EditorWindow` is a class that represents a standalone, dockable window within the Unity Editor. The process involves three main steps: creating a class that inherits from `EditorWindow`, writing a static `MenuItem` function to open the window, and implementing the `OnGUI` method to define its contents.²⁵

UI with UI Toolkit vs. ImGui

Unity offers two systems for creating editor UIs: the modern, retained-mode UI Toolkit and the older, immediate-mode GUI (ImGui). While UI Toolkit is powerful for complex UIs, ImGui (`EditorGUI` and `EditorGUILayout`) is often simpler for tool development, offers fine-grained control, and is highly compatible across Unity versions, making it an excellent starting point. This tutorial will use ImGui.

Project: "Quick Asset Pinner" Window

This project will create a simple window to which users can drag and pin frequently used assets.

1. Create the Editor Window Script:

```
C#
// Place in an "Editor" folder
using UnityEngine;
using UnityEditor;
using System.Collections.Generic;

public class QuickAssetPinner : EditorWindow
{
    private List<Object> pinnedAssets = new List<Object>();
    private Vector2 scrollPosition;

    public static void ShowWindow()
    {
        GetWindow<QuickAssetPinner>("Asset Pinner");
    }

    private void OnGUI()
    {
        GUILayout.Label("Drag Assets Here to Pin", EditorStyles.boldLabel);

        // Create a drop area
        Rect dropArea = GUILayoutUtility.GetRect(0.0f, 50.0f, GUILayout.ExpandWidth(true));
        GUI.Box(dropArea, "Drop Area");

        HandleDragAndDrop(dropArea);

        // Display pinned assets
        scrollPosition = EditorGUILayout.BeginScrollView(scrollPosition);
        for (int i = 0; i < pinnedAssets.Count; i++)
        {
            EditorGUILayout.BeginHorizontal();
            pinnedAssets[i] = EditorGUILayout.ObjectField(pinnedAssets[i], typeof(Object),
false);
```

```

        if (GUILayout.Button("X", GUILayout.Width(20)))
        {
            pinnedAssets.RemoveAt(i);
        }
        EditorGUILayout.EndHorizontal();
    }
    EditorGUILayout.EndScrollView();
}

```

```

private void HandleDragAndDrop(Rect dropArea)
{
    Event currentEvent = Event.current;
    if (!dropArea.Contains(currentEvent.mousePosition))
        return;

```

```

    if (currentEvent.type == EventType.DragUpdated |

```

```

| currentEvent.type == EventType.DragPerform)
{
    DragAndDrop.visualMode = DragAndDropVisualMode.Copy;

```

```

        if (currentEvent.type == EventType.DragPerform)
        {
            DragAndDrop.AcceptDrag();
            foreach (Object draggedObject in DragAndDrop.objectReferences)
            {
                if (!pinnedAssets.Contains(draggedObject))
                {
                    pinnedAssets.Add(draggedObject);
                }
            }
        }
        currentEvent.Use();
    }
}
}
...

```

2. **Data Persistence:** The current implementation loses its data when the editor closes. To make it persistent, the list of assets should be stored in a ScriptableObject. A single instance of this ScriptableObject can be created and loaded via AssetDatabase. This is the professional standard for saving editor tool data, as it integrates seamlessly with Unity's asset system and version control.

Chapter 9: Advanced Techniques and Expansion

Goal

This final chapter provides a roadmap for building upon the foundational skills, connecting them directly to the advanced techniques observed in the v-series.

Extending the Hierarchy and Project Windows

As analyzed in Part I, the key to customizing these windows lies in the EditorApplication delegates. A simple example demonstrates the concept:

C#

```
// Place in an "Editor" folder
using UnityEngine;
using UnityEditor;

[InitializeOnLoad]
public static class HierarchyHighlighter
{
    static HierarchyHighlighter()
    {
        EditorApplication.hierarchyWindowItemOnGUI += HandleHierarchyItemGUI;
    }
}
```

```

private static void HandleHierarchyItemGUI(int instanceID, Rect selectionRect)
{
    GameObject go = EditorUtility.InstanceIDToObject(instanceID) as GameObject;
    if (go != null && go.CompareTag("Manager"))
    {
        // Draw a red box next to any object tagged "Manager"
        Rect colorRect = new Rect(selectionRect.xMax - 20, selectionRect.y, 20,
selectionRect.height);
        EditorGUI.DrawRect(colorRect, Color.red);
    }
}
}

```

This script, which runs automatically when the editor loads, draws a red square next to any GameObject tagged "Manager." This is the fundamental technique used by vHierarchy and vFolders to add their visual flair.

Drawing in the Scene View

Building on the analysis of vRuler, this section introduces the SceneView.duringSceneGui delegate and the Handles class. The following script draws a label above the currently selected object in the scene:

C#

```

// Place in an "Editor" folder
using UnityEngine;
using UnityEditor;

[InitializeOnLoad]
public static class SceneViewLabel
{
    static SceneViewLabel()
    {
        SceneView.duringSceneGui += OnSceneGUI;
    }
}

```

```
private static void OnSceneGUI(SceneView sceneView)
{
    if (Selection.activeGameObject != null)
    {
        Handles.Label(Selection.activeGameObject.transform.position + Vector3.up * 2,
            Selection.activeGameObject.name);
    }
}
}
```

This demonstrates the basic principle for creating custom in-scene visualizations and tools.

Principles for a Cohesive Tool Suite

The v-series excels because its components feel like a single, unified product. This is achieved through key architectural principles:

- **Shared Visual Language:** Icons and colors are consistent across tools.
- **Centralized Data:** A shared ScriptableObject or configuration system can store settings for all tools, allowing them to interact and share data seamlessly.
- **Consistent UX:** Using similar shortcuts, interaction patterns (e.g., Alt-Click), and UI styles makes the entire suite feel intuitive.

Conclusion: Cultivating a Bespoke Development Environment

Summary of Learnings

This report has provided an exhaustive analysis of the KUBACHO LAB v-series, a suite of tools built on a philosophy of "Pragmatic Enhancement." Each tool—from vHierarchy's visual organization to vInspector's attribute-driven UI and vTabs' browser-like navigation—is designed to be a lightweight, intuitive, and performant solution to common workflow friction in

the Unity Editor. The technical deep dives have revealed how these polished tools are constructed using core editor scripting APIs, such as editor delegates, custom editors, property drawers, and the Handles class. Part II leveraged this analysis to provide a foundational education in these same techniques, empowering developers to move beyond being users of tools to becoming creators of them.

Buy vs. Build: An Informed Decision

The question of whether to buy an asset or build a custom solution is central to professional game development. The v-series makes a compelling case for purchasing established, well-supported tools. The time, effort, and expertise required to replicate the stability, feature set, and polish of these assets would far exceed their modest cost for any commercial project.⁴

The purpose of learning editor scripting is not to reinvent these wheels. Rather, it is to gain the ability to create bespoke tools that solve the unique, project-specific problems that no off-the-shelf asset can address. Perhaps a project requires a custom tool to validate level design data, a specialized window for managing narrative content, or a unique scene gizmo for visualizing AI behavior. Armed with the principles outlined in this report, a developer can now build those specific solutions, creating a truly tailored and maximally efficient development environment.

The Path Forward

The journey into editor scripting is a deep and rewarding one. To continue this path, developers are encouraged to explore the following resources:

- **Official Unity Documentation:** The ultimate source of truth for the UnityEditor API.
- **Official v-Series Documentation:** The developer provides excellent, detailed documentation for their tools, which serve as a benchmark for good practice.
 - vHierarchy 2 Documentation: <https://kubacho-lab.gitbook.io/vhierarchy-2/>⁸
 - vInspector 2 Documentation: <https://kubacho-lab.gitbook.io/vinspector2/>⁸
- **Unity Learn:** The official learning platform has numerous tutorials on editor scripting, from beginner to advanced topics.
- **Community Forums and Discussions:** Platforms like the Unity Forums and Reddit are invaluable for asking questions and seeing how other developers solve complex editor scripting challenges.

By combining the power of high-quality assets like the v-series with the ability to craft custom solutions, developers can transform the Unity Editor from a generic game engine into a personalized and highly optimized production powerhouse.

Works cited

1. kubacho lab, accessed on August 31, 2025, <https://kubacho.github.io/kubacho-lab/>
2. kubacho lab - Unity Asset Store, accessed on August 31, 2025, <https://assetstore.unity.com/publishers/80066>
3. kubacho lab - Unity Asset Store, accessed on August 31, 2025, <https://assetstore-fallback.unity.com/publishers/80066?locale=zh-CN>
4. Is it necessary to purchase utility tools like vInspector 2? : r/Unity3D - Reddit, accessed on August 31, 2025, https://www.reddit.com/r/Unity3D/comments/1mrq38f/is_it_necessary_to_purchase_utility_tools_like/
5. vInspector 2 | Game Content Shopper – Unity Asset Store™ Sales and Price Drops, accessed on August 31, 2025, <https://gamecontentshopper.com/asset/all-assets/vinspector-2-3/2024/07/24/>
6. vHierarchy 2.0 - Game Asset Deals, accessed on August 31, 2025, <https://www.gameassetdeals.com/asset/251320/vhierarchy-2>
7. I gave the project window a facelift. Meet my new tool - vFolders 2! : r/Unity3D - Reddit, accessed on August 31, 2025, https://www.reddit.com/r/Unity3D/comments/1fqskug/i_gave_the_project_window_a_facelift_meet_my_new/
8. kubacho-lab - Reddit, accessed on August 31, 2025, <https://www.reddit.com/user/kubacho-lab/>
9. I gave the hierarchy window a UX boost. Check out my new tool - vHierarchy 2! : r/Unity3D, accessed on August 31, 2025, https://www.reddit.com/r/Unity3D/comments/1i1x6e4/i_gave_the_hierarchy_window_a_ux_boost_check_out/
10. vInspector 2 | Utilities Tools - Unity Asset Store, accessed on August 31, 2025, <https://assetstore.unity.com/packages/tools/utilities/vinspector-2-252297>
11. vTabs 2.0 - Game Asset Deals, accessed on August 31, 2025, <https://www.gameassetdeals.com/asset/263645/vtabs-2>
12. This asset is a must for your hierarchy in Unity #Unity3D #unitytips #unityassetstore - YouTube, accessed on August 31, 2025, <https://www.youtube.com/shorts/mNU8r9H9rnQ>
13. vFavorites 2 | Game Content Shopper – Unity Asset Store™ Sales and Price Drops, accessed on August 31, 2025, <https://gamecontentshopper.com/asset/all-assets/vfavorites-2-3/2024/09/18/>
14. Unity EDITOR Will NEVER Be The Same (vInspector 2) - YouTube, accessed on August 31, 2025, <https://www.youtube.com/watch?v=26AslXmvU4M>
15. I got a bit carried away tweaking the inspector... so here's my new tool, vInspector 2! : r/Unity3D - Reddit, accessed on August 31, 2025,

https://www.reddit.com/r/Unity3D/comments/1ddqkd4/i_got_a_bit_carried_away_tweaking_the_inspector/

16. Odin Inspector and Serializer | Improve your workflow in Unity, accessed on August 31, 2025, <https://odininspector.com/>
17. Best Practices | Odin Inspector for Unity, accessed on August 31, 2025, <https://odininspector.com/tutorials/serialize-anything/best-practices>
18. Odin Inspector - Yes or No? : r/Unity3D - Reddit, accessed on August 31, 2025, https://www.reddit.com/r/Unity3D/comments/1hzojsl/odin_inspector_yes_or_no/
19. Editor Enhancers Bundle | Game Content Shopper – Unity Asset Store™ Sales and Price Drops, accessed on August 31, 2025, <https://gamecontentshopper.com/asset/all-assets/editor-enhancers-bundle/2023/06/22/>
20. What are your Unity Asset Store favorites? - Creator Feedback - VRChat Ask Forum, accessed on August 31, 2025, <https://ask.vrchat.com/t/what-are-your-unity-asset-store-favorites/23387>
21. vTabs | Game Content Shopper – Unity Asset Store™ Sales and Price Drops, accessed on August 31, 2025, <https://gamecontentshopper.com/asset/all-assets/vtabs-2/2023/05/10/>
22. vRuler | Utilities Tools - Unity Asset Store, accessed on August 31, 2025, <https://assetstore.unity.com/packages/tools/utilities/vruler-249760>
23. Creating a Custom Unity Button with Editor Extension | by Code_With_K | Medium, accessed on August 31, 2025, https://medium.com/@Code_With_K/creating-a-custom-unity-button-with-editor-extension-2650f2a6abda
24. Unity Custom Inspectors Tutorial: Getting Started - Kodeco, accessed on August 31, 2025, <https://www.kodeco.com/7751-unity-custom-inspectors-tutorial-getting-started>
25. Introduction to Editor Scripting - Unity Learn, accessed on August 31, 2025, <https://learn.unity.com/tutorial/introduction-to-editor-scripting>
26. UnityEditor - Scripting API - Unity - Manual, accessed on August 31, 2025, <https://docs.unity3d.com/ScriptReference/UnityEditor.html>
27. Editor Scripting - Unity Learn, accessed on August 31, 2025, <https://learn.unity.com/tutorial/editor-scripting>
28. Learn Unity Editor Scripting: Property Drawers (Part 2) | by No Such Dev | Medium, accessed on August 31, 2025, <https://nosuchstudio.medium.com/learn-unity-editor-scripting-property-drawers-part-2-6fe6097f1586>
29. Scripting API: PropertyDrawer - Unity - Manual, accessed on August 31, 2025, <https://docs.unity3d.com/ScriptReference/PropertyDrawer.html>
30. Create a custom Editor window with C# script - Unity - Manual, accessed on August 31, 2025, <https://docs.unity3d.com/6000.1/Documentation/Manual/UIE-HowTo-CreateEditorWindow.html>